

COMSATS UNIVERSITY ISLAMABAD

WAH CAMPUS

Department of Computer Science

DevOps Lab Task

Course Title: DevOps

Course Code: CSC323

Instructor Name: Sir Ikram

Class: BSE-8A

Name	Registration No.
Muhammad Sami	SP23-BSE-039

Containerization, Docker Volume & Docker Network on EC2

This report documents a DevOps lab exercise in which a three-tier Laravel "Student Management System" (Nginx frontend, PHP-FPM backend, MySQL database) is built and deployed directly on an AWS EC2 instance, rather than a local virtual machine. The lab also verifies two key Docker concepts on this cloud host: that named Docker volumes keep data safe even if a container is deleted and recreated, and that containers on the same Compose network can reach each other by service name. Finally, the built images are published to Docker Hub so they can be reused or deployed elsewhere. Each step below first explains the activity and the command used, and is then followed by the terminal or browser screenshot captured as evidence of that step.

Part A: Preparing the AWS EC2 Environment

Before any application code is written, the target cloud server is confirmed to be running and reachable, and the tools already installed on it are checked.

Verifying the EC2 Instance

The AWS Management Console is used to confirm the EC2 instance that will host the application is running and to note down its network details.

What Is Being Done: The student opens the EC2 console and inspects the instance summary for the virtual machine that will be used for this lab, checking its instance ID, state, and IP addresses before connecting to it.

Observed Result: The instance summary confirms the instance (ID i-050ab15419f29f01c, name "samivmtwo", type t3.micro) is in the Running state, with public IP address 13.51.101.222 and private IP 172.31.19.94. This public IP is the address that will later be used to reach the deployed application from a browser.

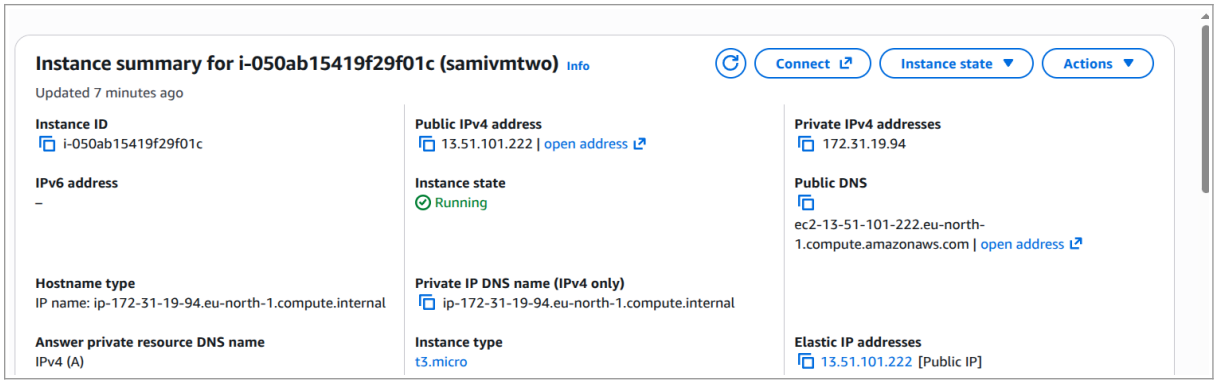


Figure 1: AWS EC2 console confirming the target instance is running, with its public and private IP addresses.

Connecting to the Instance and Confirming Docker

With the instance confirmed to be running, the student connects to it over SSH and checks whether Docker is already installed.

Command Used:

```
ssh -i awskey.pem ubuntu@<public-ip>
docker --version
```

Command Explanation:

- ssh -i awskey.pem ubuntu@<public-ip> → opens a secure shell session to the EC2 instance, authenticating with the private key file (awskey.pem) associated with the instance's key pair, logging in as the default ubuntu user.
- docker --version → prints the installed Docker Engine version, confirming Docker is already available on this instance.

What Is Being Done: The student logs into the remote EC2 server from their local machine and checks the pre-installed software before starting the containerization work.

Observed Result: The login banner shows standard Ubuntu Pro update notices, and docker --version reports Docker version 29.1.3, confirming Docker Engine is already installed on this instance and ready to use.

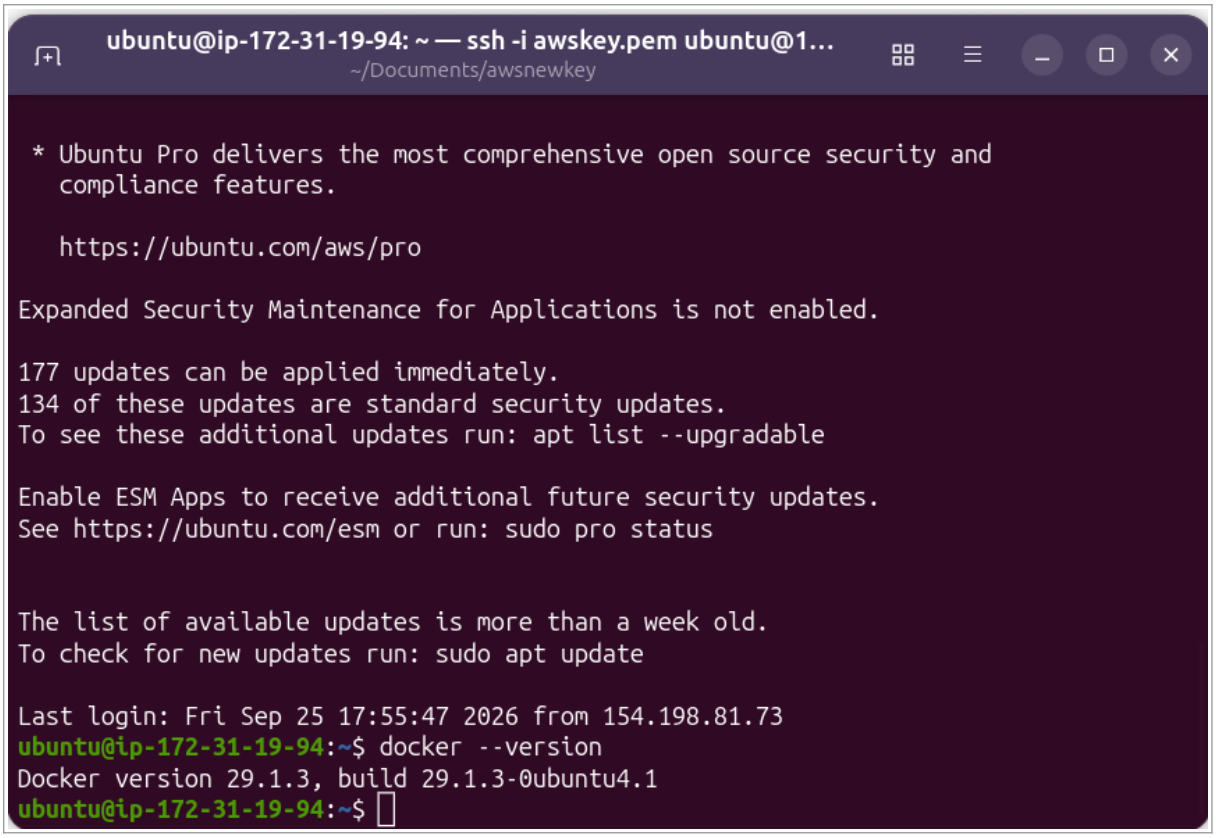


Figure 2: Connecting to the EC2 instance over SSH and confirming Docker is installed.

Part B: Building the 3-Tier Laravel Application on the EC2 Host

Scaffolding the Laravel Project

Unlike a setup that uses a throwaway Composer container, here Composer is used directly on the EC2 host to create the Laravel project, after first confirming Docker Compose is also available and creating

a dedicated project folder.

Command Used:

```
sudo docker compose version
mkdir -p ~/laravel-3tier
cd ~/laravel-3tier
pwd
composer create-project laravel/laravel .
```

Command Explanation:

- sudo docker compose version → confirms the Docker Compose plugin (v2.40.3) is also available on this instance.
- mkdir -p ~/laravel-3tier → creates the project directory that will hold all of the application and Docker configuration files.
- cd ~/laravel-3tier and pwd → moves into the new directory and confirms its full path (/home/ubuntu/laravel-3tier).
- composer create-project laravel/laravel . → downloads the Laravel framework and its dependencies directly into the current directory using the Composer package manager installed on the host.

What Is Being Done: The student is preparing a clean, dedicated project directory on the EC2 host and generating a brand-new Laravel application skeleton directly on that server, which will later be packaged into a Docker image.

Observed Result: Docker Compose reports version 2.40.3, the laravel-3tier directory is created at /home/ubuntu/laravel-3tier, and Composer begins creating the "laravel/laravel" project, installing version 13.10.1 and starting to resolve and lock all of its dependencies.

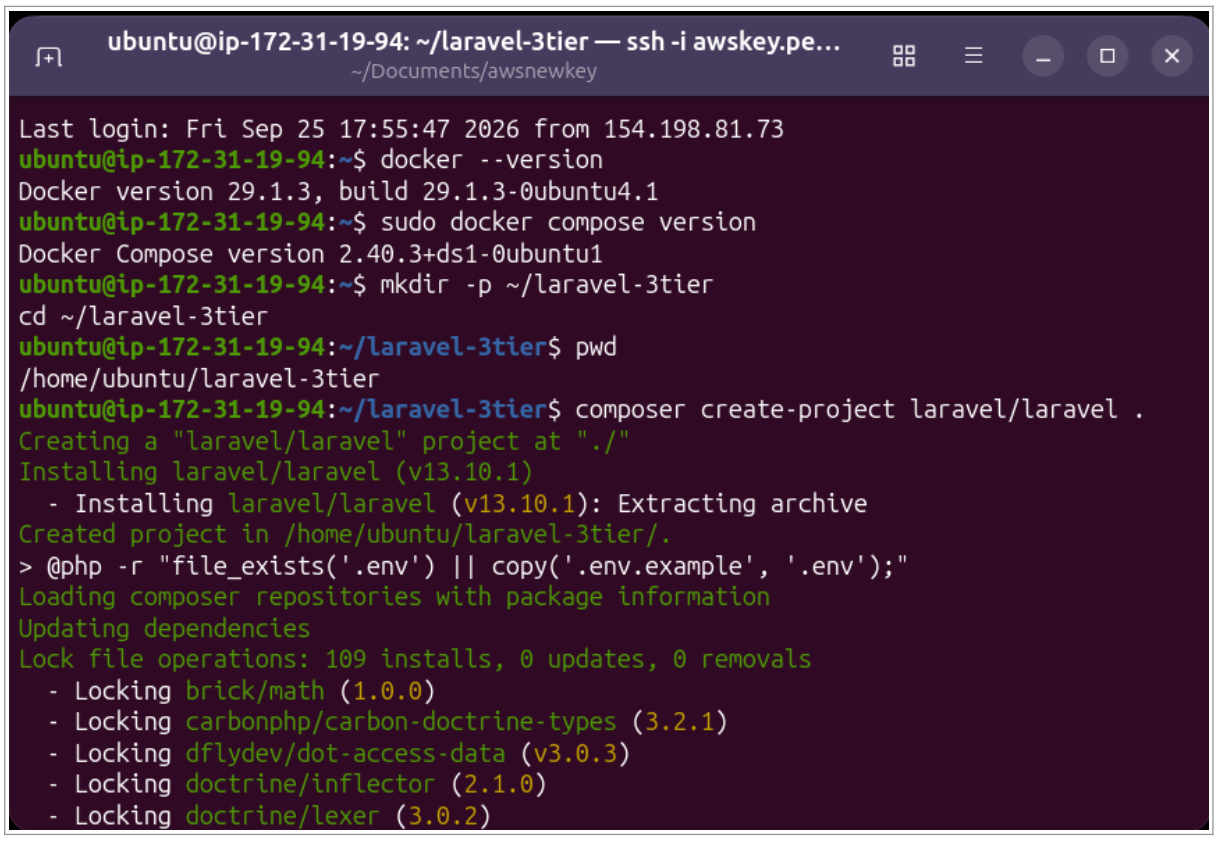


Figure 3: Confirming Docker Compose, creating the project directory, and starting the Laravel installation

with Composer.

Composer continues resolving dependencies and, once installation finishes, automatically runs Laravel's post-install scripts — discovering installed packages and generating the application's unique encryption key.

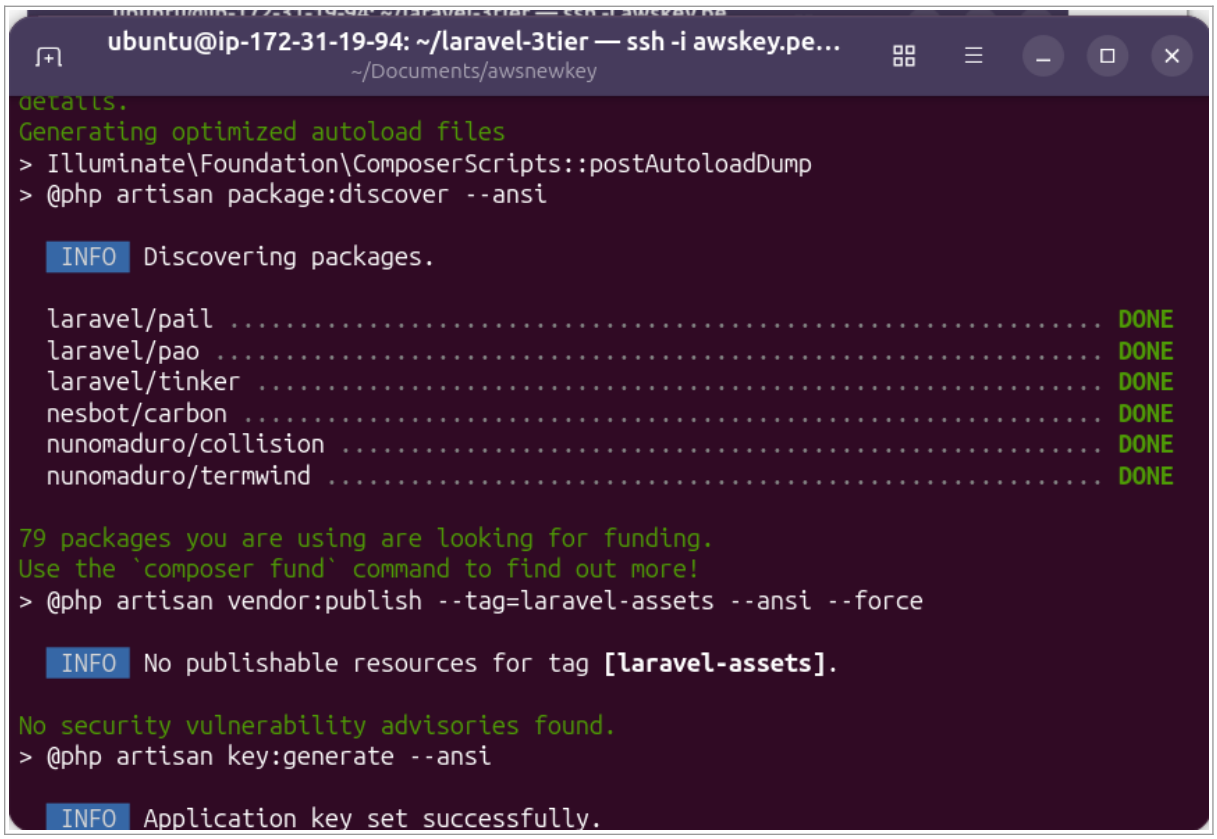


Figure 4: Composer finishing package discovery and generating the application key with php artisan key:generate.

Composer finishes by preparing the local SQLite placeholder database and running Laravel's built-in framework migrations, confirming the project scaffold is fully ready inside ~/laravel-3tier.



Figure 5: Laravel's initial framework migrations completing, confirming the project scaffold is ready.

Configuring the Environment File for MySQL

Laravel needs to be told how to reach the MySQL database that will run in its own container, so the environment configuration file is edited next.

Command Used:

```
nano .env
```

Command Explanation:

- DB_CONNECTION=mysql → tells Laravel to use the MySQL database driver instead of the default SQLite.
- DB_HOST=db → sets the database hostname to "db", which will be the service name given to the MySQL container in the Compose file, allowing Docker's internal DNS to resolve it.
- DB_PORT=3306, DB_DATABASE=laravel, DB_USERNAME=laravel, DB_PASSWORD=password → set the standard MySQL port and the database name, username, and password that the MySQL container will be configured with.

What Is Being Done: The student is pointing the Laravel application at the MySQL service it will connect to once the full Docker Compose stack is running, instead of the default local SQLite file.

Observed Result: The .env file is updated with the mysql connection settings shown above, which must match the credentials that will later be given to the MySQL container in compose.yaml.

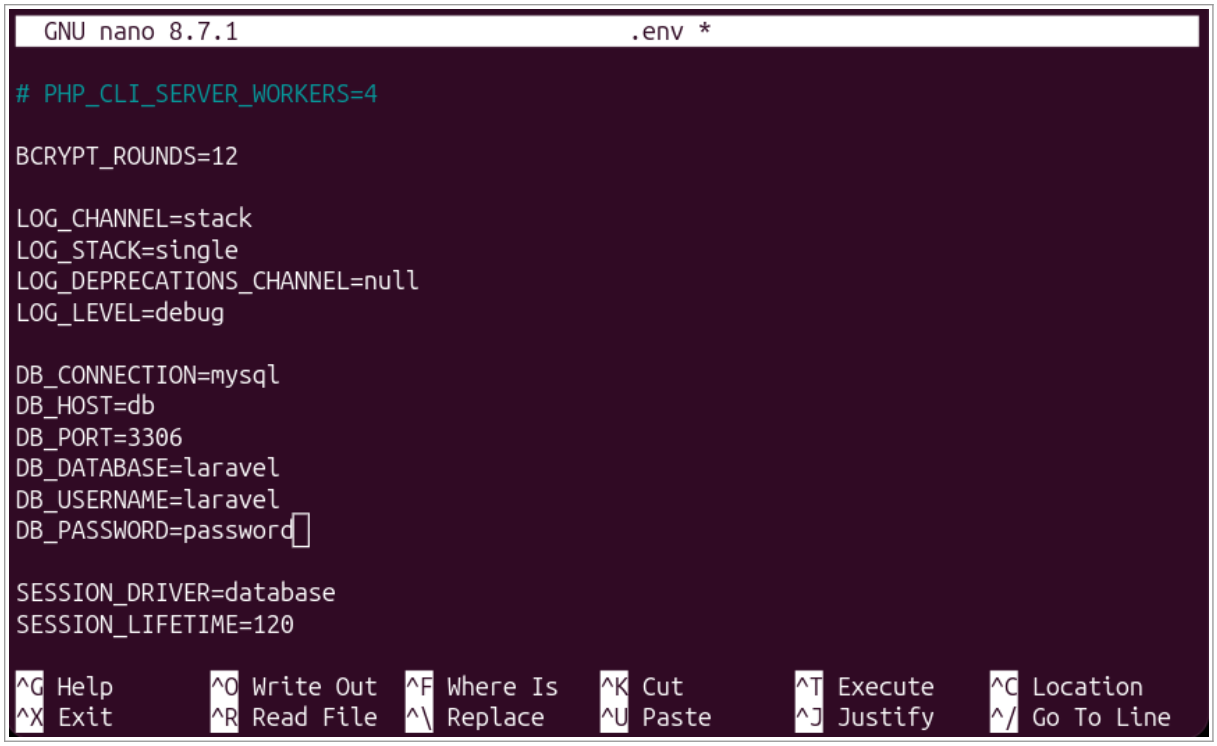


Figure 6: Editing the .env file to configure the Laravel application to use the MySQL database service.

Creating the Student Management Feature

With Laravel scaffolded and configured, the student adds the application feature used throughout the rest of the lab: a simple Student Management System with a model, controller, database migration, view, and route.

First, the Student Eloquent model is created, defining which fields (name and email) can be mass-assigned.

```
  GNU nano 8.7.1                    app/Models/Student.php *
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Model;

class Student extends Model
{
    protected $fillable = [
        'name',
        'email',
    ];
}
```

Figure 7: Creating the Student model in app/Models/Student.php.

Next, the StudentController is written, with an index() method that loads all students for display and a store() method (further down in the file) that validates and saves a new student submitted through the form.

```
  GNU nano 8.7.1          app/Http/Controllers/StudentController.php *
<?php

namespace App\Http\Controllers;

use App\Models\Student;
use Illuminate\Http\Request;

class StudentController extends Controller
{
    public function index()
    {
        $students = Student::all();

        return view('students', compact('students'));
    }

    public function store(Request $request)
    {
        $request->validate([
```

Figure 8: Creating the StudentController with index() and store() methods.

A new migration file is created to define the students table structure: an auto-incrementing ID, a name field, and a unique email field.

```
database/migrations/2026_09_28_000000_create_students_table.php *
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    public function up(): void
    {
        Schema::create('students', function (Blueprint $table) {
            $table->id();
            $table->string('name');
            $table->string('email')->unique();
            $table->timestamps();
        });
    }

    public function down(): void
```

Figure 9: Writing the create_students_table migration with name and unique email columns.

The Blade view is written next, containing the HTML form fields for the student's name and email, and a submit button that posts to the store route.

```
ubuntu@ip-172-31-19-94: ~/laravel-3tier — ssh -i awskey.pe...
~/Documents/awsnewkey
GNU nano 8.7.1              resources/views/students.blade.php *
    @csrf

    <input
        type="text"
        name="name"
        placeholder="Student Name"
        required
    >

    <input
        type="email"
        name="email"
        placeholder="Student Email"
        required
    >

    <button type="submit">Add Student</button>

</form>
```

Figure 10: Writing the add-student form in resources/views/students.blade.php.

Finally, the application routes are defined: a GET route for the home page that shows the index view, and a POST route that sends new form submissions to the controller's store method.

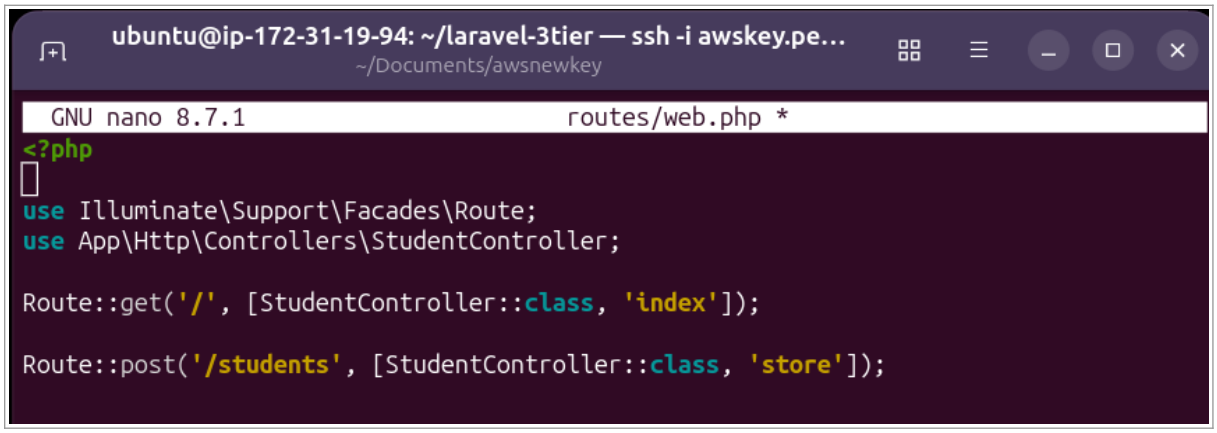


Figure 11: Defining the index and store routes in routes/web.php.

Before moving on to Docker, the student reviews the shell history to confirm every feature file was created in the right place.

Command Used:

```
nano .env
mkdir -p app/Http/Controllers
nano app/Models/Student.php
nano app/Http/Controllers/StudentController.php
nano database/migrations/2026_09_28_000000_create_students_table.php
nano resources/views/students.blade.php
nano routes/web.php
```

Command Explanation:

- Each nano command opens a text editor session on the named file, creating it if it does not already exist.
- mkdir -p app/Http/Controllers → ensures the controllers folder exists before a new controller file is written into it.

What Is Being Done: The student confirms, from the accumulated shell history, that all of the pieces of the Student Management feature (environment config, model, controller, migration, view, and routes) were created in sequence.

Observed Result: The command history lists every file exactly as created in the previous steps, with no errors, confirming the feature's source files are all in place before the Docker images are built.

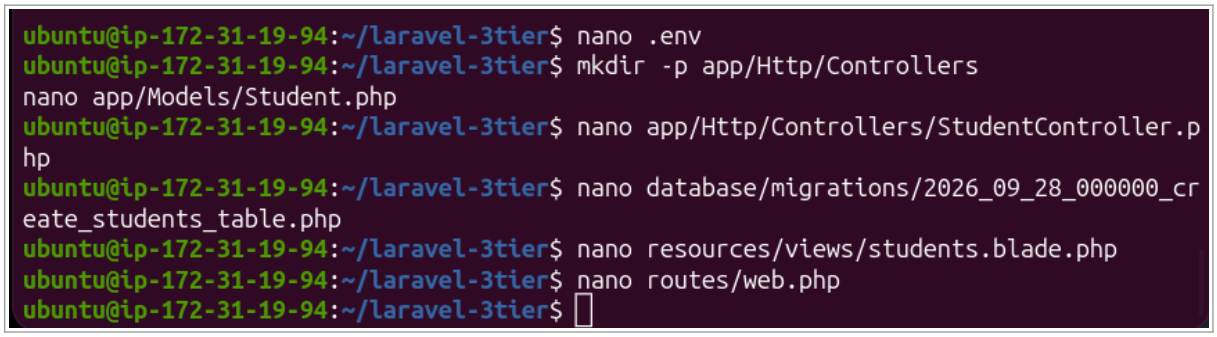


Figure 12: Shell history confirming all Student Management feature files were created successfully.

Writing the Backend Dockerfile

With the application code complete, a Dockerfile is written to package the Laravel backend into a PHP-FPM image, exactly as required to run the application in a container.

Command Used:

```
FROM php:8.4-fpm

WORKDIR /var/www/html

RUN apt-get update && apt-get install -y \
 git \
 unzip \
 libzip-dev \
 libpng-dev \
 libonig-dev \
 libxml2-dev \
 && docker-php-ext-install \
 pdo_mysql \
 mbstring \
 bcmath \
 exif \
 pcntl \
 zip \
 && rm -rf /var/lib/apt/lists/*
```

Command Explanation:

- FROM php:8.4-fpm → uses the official PHP 8.4 FastCGI Process Manager image as the base for the backend container.
- WORKDIR /var/www/html → sets the standard web-root directory as the working directory for later instructions.
- apt-get install … → installs the system libraries Laravel's PHP extensions depend on (zip, PNG, onig, XML).
- docker-php-ext-install pdo_mysql mbstring bcmath exif pcntl zip → compiles and enables the PHP extensions Laravel requires, most importantly pdo_mysql for talking to the MySQL container.
- rm -rf /var/lib/apt/lists/* → clears the package index cache to keep the image smaller.

What Is Being Done: The student is authoring the backend image definition, choosing the PHP-FPM base image and enabling exactly the PHP extensions the Laravel application needs to run and to talk to MySQL.

Observed Result: The nano editor shows the completed Dockerfile with the correct base image and extension list, ready to be built.

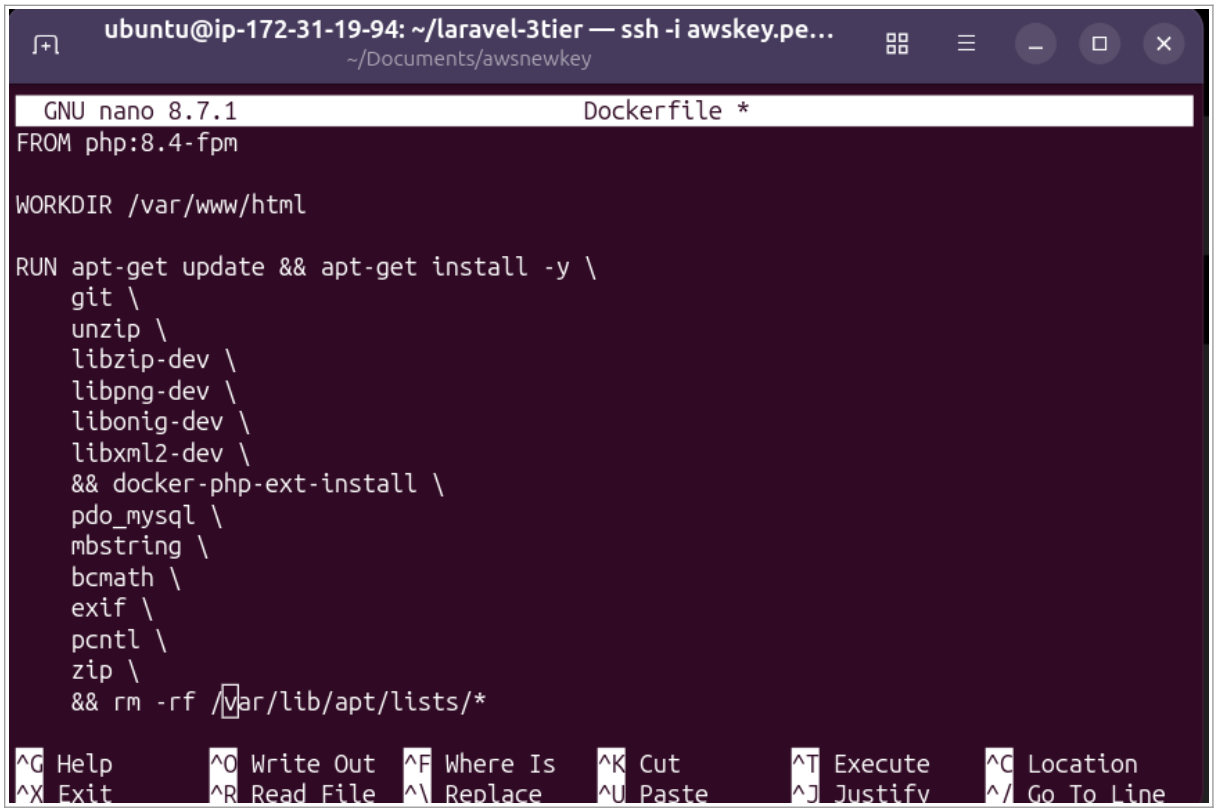


Figure 13: Writing the backend Dockerfile, based on the php:8.4-fpm image.

Creating the Nginx Configuration and Frontend Dockerfile

To serve the application over HTTP, a lightweight Nginx frontend image is defined alongside its own configuration file that forwards PHP requests to the backend container.

Command Used (nginx/Dockerfile):

```
FROM nginx:alpine
COPY nginx/default.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

Command Explanation:

- FROM nginx:alpine → uses the small, official Alpine-based Nginx image as the base for the frontend container.
- COPY nginx/default.conf /etc/nginx/conf.d/default.conf → replaces Nginx's default site configuration with the project's own configuration file.
- EXPOSE 80 → documents that the container listens for HTTP traffic on port 80.

What Is Being Done: The student defines a minimal frontend image whose only job is to serve static files and forward PHP requests on to the backend container.

Observed Result: The nano editor shows the completed three-line nginx/Dockerfile, ready to be built as the laravel-3tier-frontend image.

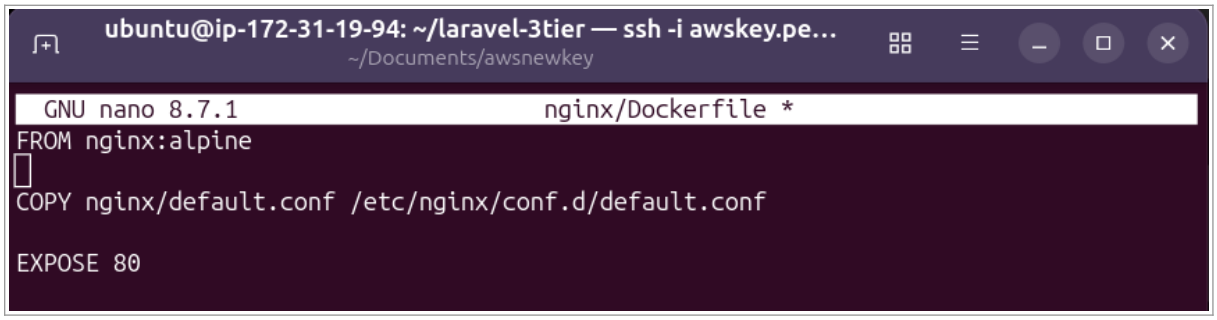


Figure 14: Writing nginx/Dockerfile, based on the lightweight nginx:alpine image.

The matching Nginx site configuration is then written, defining how incoming requests are routed.

Command Used (nginx/default.conf):

```
server {
 listen 80;
 server_name localhost;
 root /var/www/html/public;
 index index.php index.html;

 location / {
 try_files $uri $uri/ /index.php?$query_string;
 }

 location ~ \.php$ {
 try_files $uri =404;
 fastcgi_pass backend:9000;
 fastcgi_index index.php;
 include fastcgi_params;
 fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
 }

 location ~ /\.ht {
 deny all;
 }
}
```

Command Explanation:

- root /var/www/html/public → serves files from Laravel's public directory, the framework's standard web root.
- location / { try_files … } → routes all normal requests through Laravel's front controller (index.php) if a matching static file is not found.
- location ~ \.php$ { … fastcgi_pass backend:9000; … } → forwards any request for a .php file to the backend container's PHP-FPM process, listening on port 9000, using the Compose service name "backend" as the hostname.
- location ~ /\.ht { deny all; } → blocks direct access to Apache-style .htaccess files for security.

What Is Being Done: The student configures Nginx to act as the public-facing web server, handing off all PHP execution to the backend container over the internal Docker network rather than running PHP itself.

Observed Result: The completed configuration file correctly targets backend:9000 for PHP requests, which is exactly the internal service name and port the backend container will expose once the stack is running.



Figure 15: Writing the Nginx configuration, including the fastcgi_pass directive to the backend container.

The remainder of the same configuration file is shown here, completing the PHP fastcgi block and the security rule that denies access to hidden .ht files.

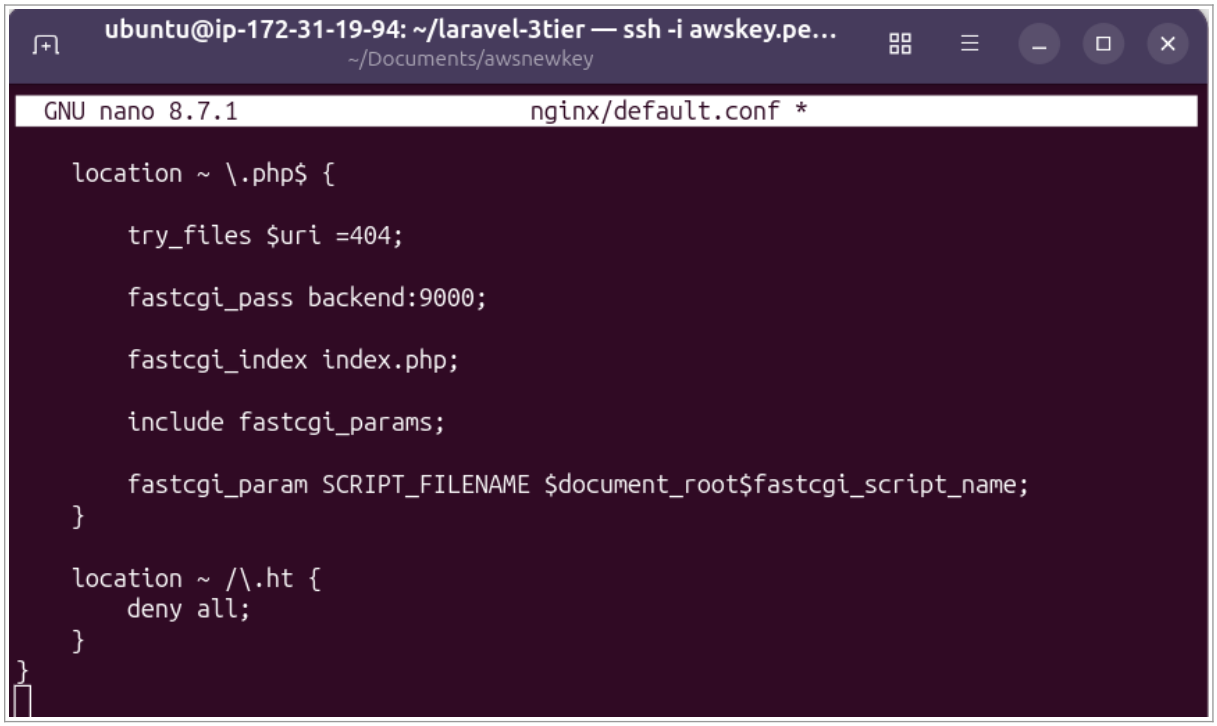


Figure 16: The remainder of nginx/default.conf, completing the fastcgi and security configuration.

The shell history confirms the order in which the backend Dockerfile and all of the Nginx files were created.

Command Used:

```
nano Dockerfile
mkdir -p nginx
nano nginx/Dockerfile
nano nginx/default.conf
```

What Is Being Done: The student confirms, from the shell history, that the backend Dockerfile, the nginx/ directory, and both Nginx configuration files were created successfully and in the correct order.

Observed Result: Each command completes without error, confirming both Docker image definitions and the Nginx configuration are ready for the images to be built.

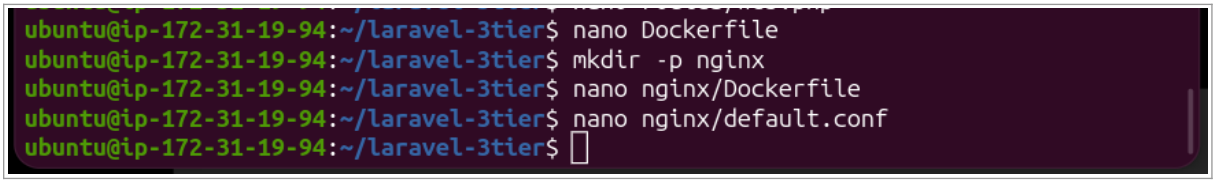


Figure 17: Shell history confirming the backend Dockerfile and Nginx configuration files were created.

Building the Backend and Frontend Images Individually

Before introducing Docker Compose, the student first builds each image manually with docker build, to confirm each Dockerfile works correctly on its own.

Command Used:

```
cd ~/laravel-3tier
sudo docker build -t laravel-3tier-backend:latest .
```

Command Explanation:

- docker build -t laravel-3tier-backend:latest . → builds a new image named laravel-3tier-backend, tagged latest, using the Dockerfile in the current directory as the build instructions and the current directory as the build context.

What Is Being Done: The student builds the backend image directly from the Dockerfile written earlier, sending the whole project folder to the Docker daemon as the build context.

Observed Result: Docker warns that the classic (legacy) builder is deprecated in favour of BuildKit, then proceeds: it sends a 56.57MB build context to the daemon and begins step 1 of 9, pulling the php:8.4-fpm base image layers.

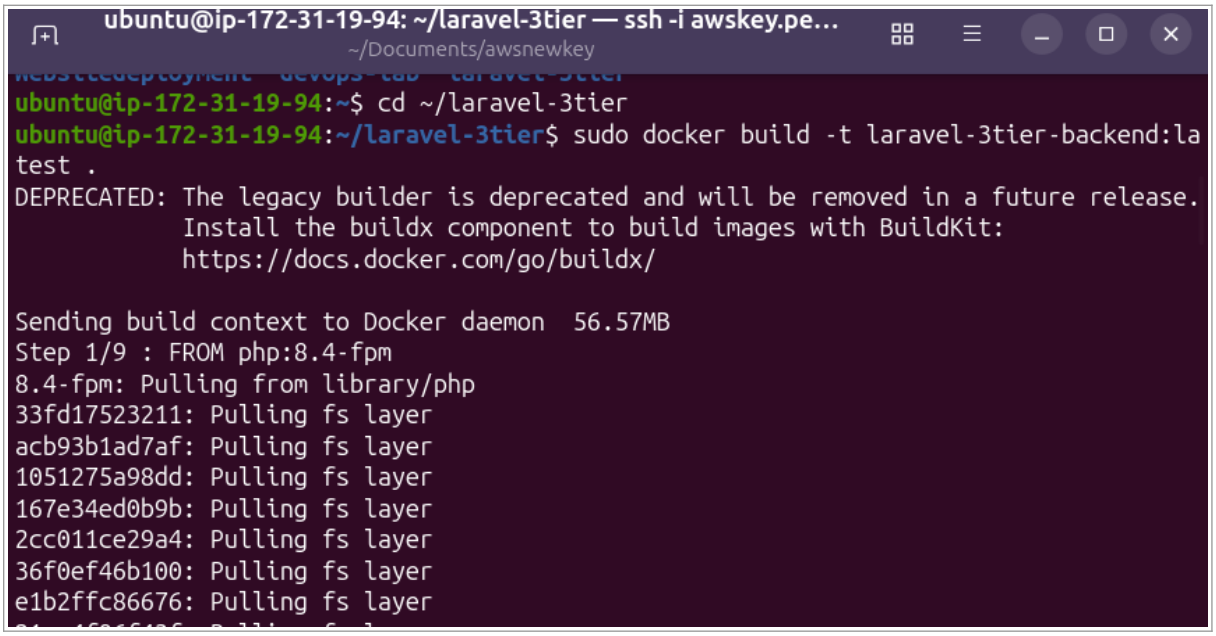


Figure 18: Building the backend image manually with docker build, using the legacy builder.

As the build progresses, Composer's dependency-discovery step runs inside the image build, installing the same Laravel packages that were installed on the host earlier.

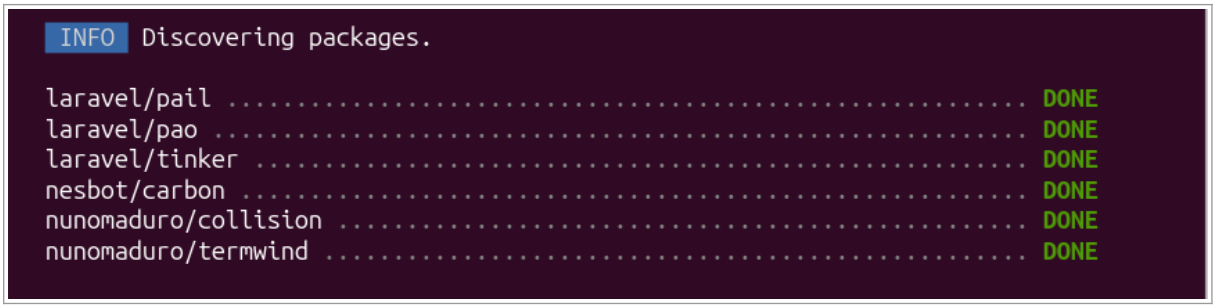


Figure 19: Composer discovering and installing packages as part of the Docker image build.

The build finishes its remaining steps — fixing file ownership, exposing port 9000, and setting php-fpm as the container's startup command — completing the backend image successfully.

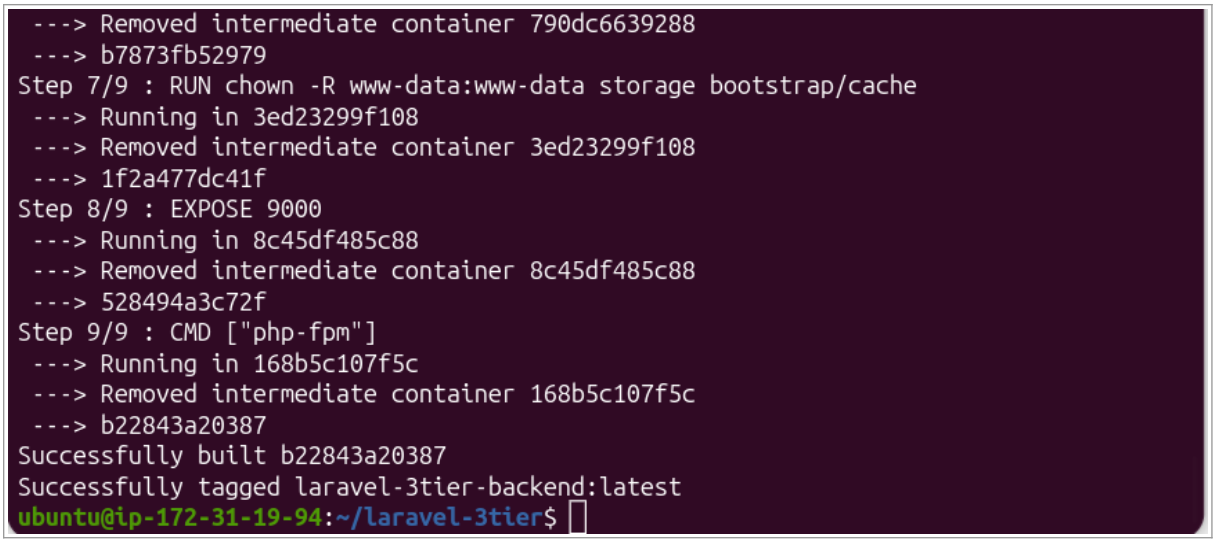


Figure 20: The backend image build finishing and being tagged laravel-3tier-backend:latest.

With the backend image built, the frontend image is built the same way, using the separate Dockerfile inside the nginx/ folder.

Command Used:

```
sudo docker build -t laravel-3tier-frontend:latest -f nginx/Dockerfile .
```

Command Explanation:

- -t laravel-3tier-frontend:latest → names and tags the resulting image.
- -f nginx/Dockerfile → tells Docker to use the Dockerfile located inside the nginx/ folder instead of the default Dockerfile in the current directory.
- . → still uses the project root as the build context, so the COPY instruction can reach nginx/default.conf.

What Is Being Done: The student builds the second custom image, this time for the Nginx frontend, using its own separate Dockerfile.

Observed Result: Docker pulls the small nginx:alpine base image, copies in the custom configuration file, and exposes port 80, successfully building and tagging the laravel-3tier-frontend:latest image.

```
Successfully tagged laravel-3tier-backend:latest
ubuntu@ip-172-31-19-94:~/laravel-3tier$ sudo docker build -t laravel-3tier-frontend:l
atest -f nginx/Dockerfile .
DEPRECATED: The legacy builder is deprecated and will be removed in a future release.
            Install the buildx component to build images with BuildKit:
            https://docs.docker.com/go/buildx/

Sending build context to Docker daemon  56.57MB
Step 1/3 : FROM nginx:alpine
alpine: Pulling from library/nginx
745dfb2690dd: Pulling fs layer
64c8194480fe: Pulling fs layer
e76228b47809: Pulling fs layer
6c53d0b2a666: Pulling fs layer
d9aae54b5831: Pulling fs layer
9a9a644fdd6a: Pulling fs layer
e72112c14215: Pulling fs layer
```

Figure 21: Building the frontend image with docker build, using the nginx/Dockerfile.

The frontend build finishes copying the configuration file and exposing port 80, completing the image.

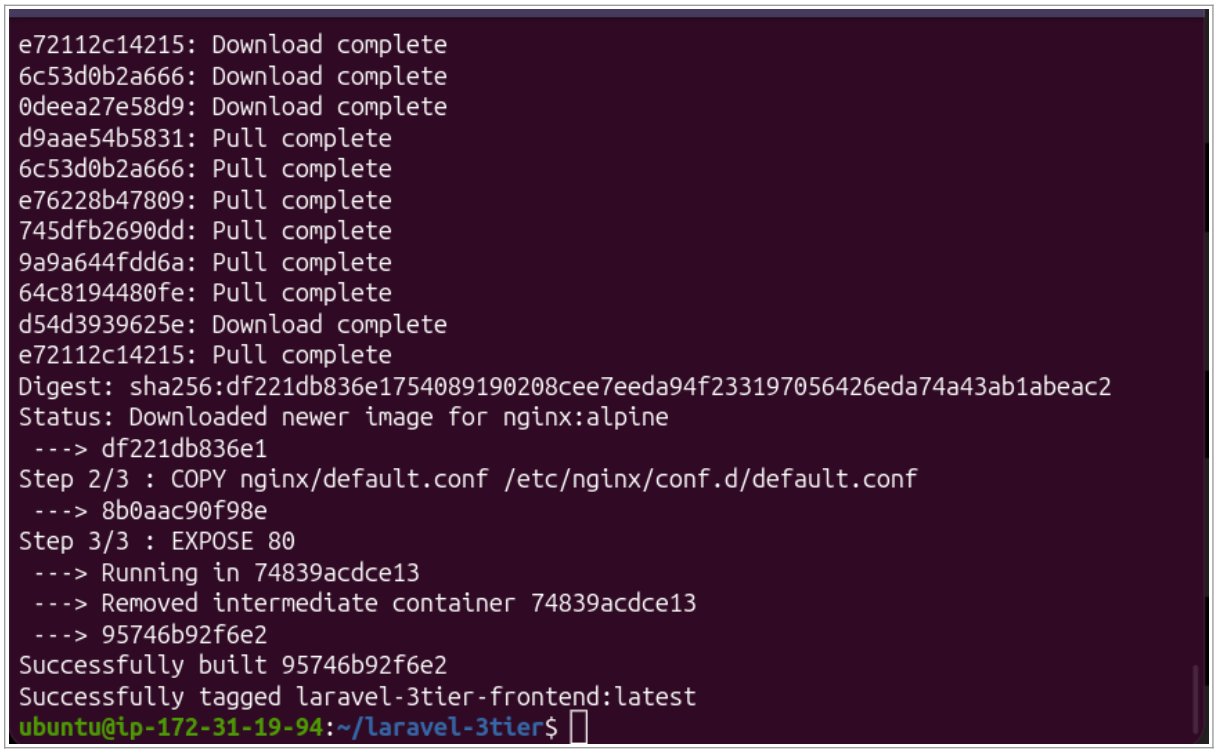


Figure 22: The frontend image build completing and being tagged laravel-3tier-frontend:latest.

With both images built, the local image list is checked to confirm they exist alongside any other images already present on the instance.

Command Used:

```
sudo docker images
```

Command Explanation:

- docker images → lists every Docker image currently stored on the host, along with its ID and disk usage.

What Is Being Done: The student verifies both custom images were created successfully before moving on to define the full stack with Docker Compose.

Observed Result: The list shows both laravel-3tier-backend:latest and laravel-3tier-frontend:latest alongside their base images (composer:2, nginx:alpine, php:8.4-fpm) and some unrelated images already present on the instance from earlier work, confirming the new images built successfully.

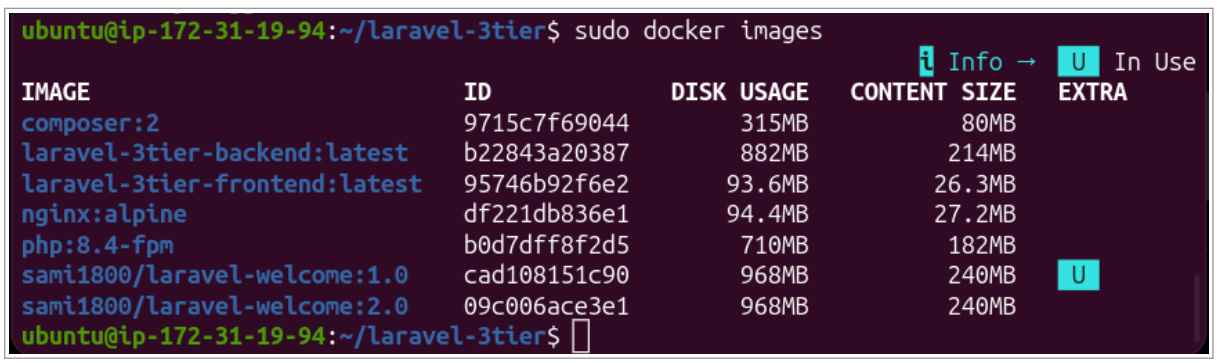


Figure 23: Confirming both custom images were built, using docker images.

Defining the Full Stack with Docker Compose

To run the frontend, backend, and database together as a single coordinated stack, a Docker Compose file is written, defining each service, its build context, ports, volumes, and dependencies.

Command Used:

```
services:

 frontend:
 build:
 context: .
 dockerfile: nginx/Dockerfile

 container_name: laravel-frontend

 ports:
 - "80:80"

 volumes:
 - backend_data:/var/www/html:ro

 depends_on:
 - backend

 networks:
 ...
```

Command Explanation:

- build: context/dockerfile → tells Compose to build the frontend image itself, from the nginx/Dockerfile, instead of pulling a pre-built image.
- container_name: laravel-frontend → gives the running container a fixed, predictable name instead of an auto-generated one.
- ports: "80:80" → maps port 80 on the EC2 host directly to port 80 inside the frontend container, which is what makes the application reachable at the instance's public IP address on the standard HTTP port.
- volumes: backend_data:/var/www/html:ro → mounts the same named volume used by the backend container into the frontend as read-only, so Nginx can serve Laravel's public assets directly from it.
- depends_on: backend → tells Compose to start the backend service before the frontend.

What Is Being Done: The student is assembling the final piece of configuration that ties the frontend, backend, and (further down in the same file) the database service together, including the port mapping that exposes the application to the public internet on port 80.

Observed Result: The nano editor shows the frontend service definition being written correctly, with the port mapping, shared volume, and startup dependency all in place.

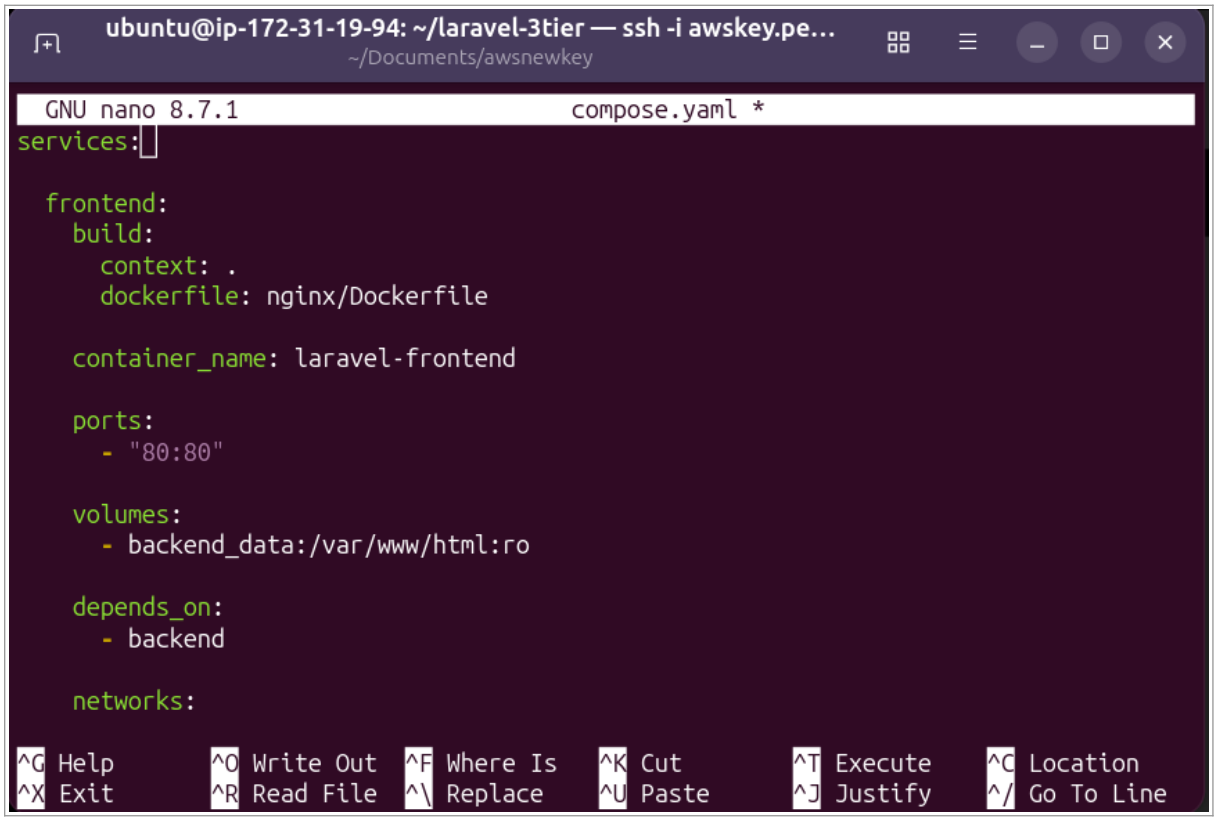


Figure 24: Writing the frontend service definition in compose.yaml, including the 80:80 port mapping.

Now that compose.yaml exists, Docker Compose is used to build the images defined in it, ensuring the images Compose will use exactly match the configuration just written.

Command Used:

```
sudo docker compose build
```

Command Explanation:

- docker compose build → builds (or rebuilds) the images for every service in compose.yaml that defines a build section.

What Is Being Done: The student rebuilds the backend and frontend images through Compose itself, using BuildKit, rather than the manual docker build commands used earlier, so that the images are properly associated with the Compose project.

Observed Result: Compose reports it is building using its newer BuildKit-based engine and proceeds through each build stage — loading the Dockerfile, pulling base image layers, and transferring the build context — for the backend service.

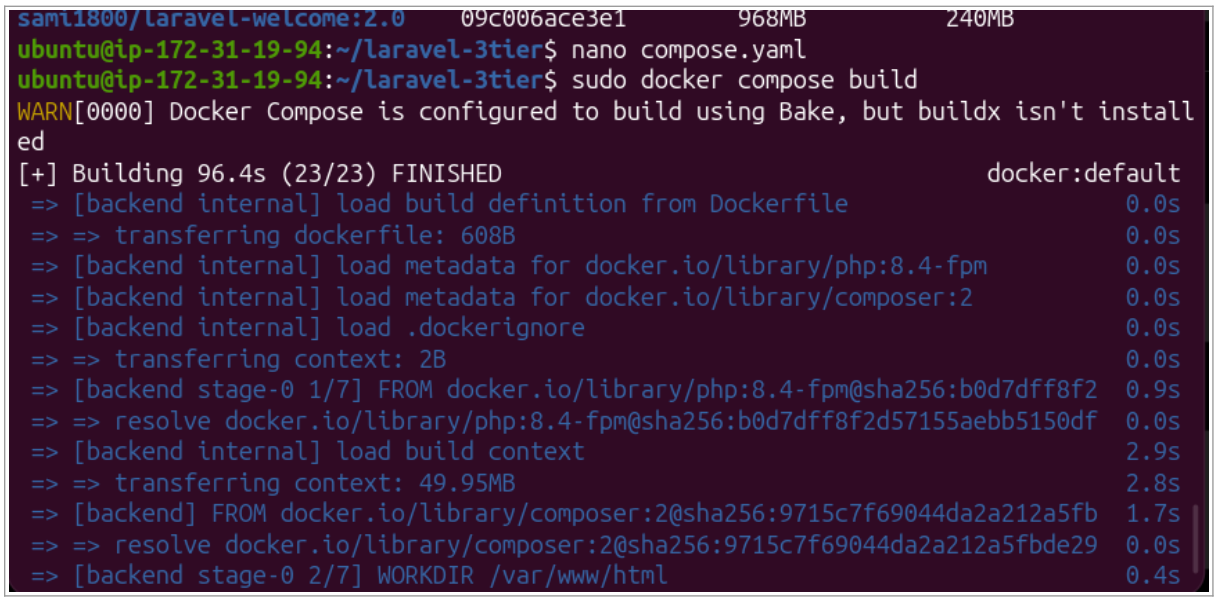


Figure 25: Rebuilding the backend and frontend images through docker compose build.

The build finishes exporting the frontend image layers and confirms both services built successfully.



Figure 26: Both the backend and frontend images reported as Built by docker compose build.

Starting the Full Stack

With both images built and registered with Compose, the entire three-tier stack is started with a single command.

Command Used:

```
sudo docker compose up -d
```

Command Explanation:

- docker compose up -d → creates and starts every service defined in compose.yaml (frontend, backend, and db) in detached (-d) mode, building any missing images and creating any missing networks or volumes automatically.

What Is Being Done: The student launches the full application stack for the first time on the EC2 instance.

Observed Result: Compose begins pulling the mysql image for the database service, showing download progress for each of its image layers.



Figure 27: Starting the full stack with docker compose up -d; the MySQL database image is pulled.

Compose continues by creating the shared network and the named volumes for the backend and database, then starts the laravel-db and laravel-backend containers.

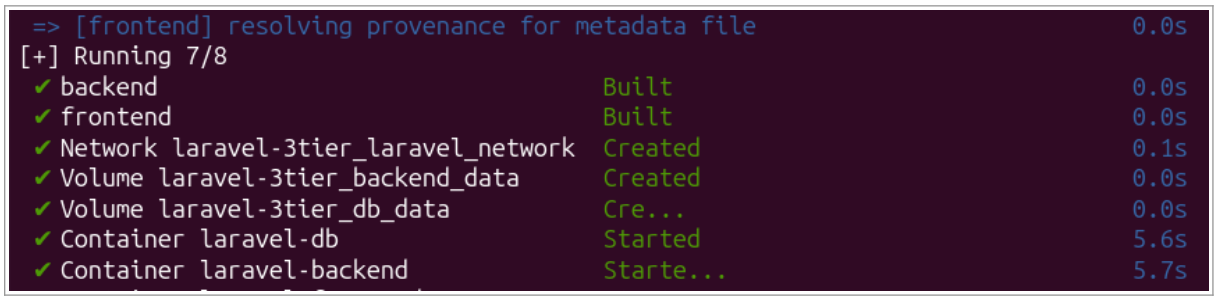


Figure 28: Docker Compose creating the network, volumes, and starting the database and backend containers.

The command finishes with all three containers — laravel-db, laravel-backend, and laravel-frontend — reported as running.

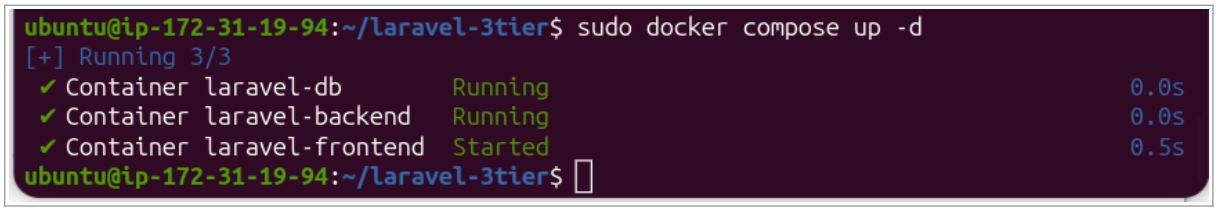


Figure 29: All three containers (db, backend, frontend) confirmed running after docker compose up -d.

Verifying the Running Containers

As a final check on the stack, the status of every service container is listed to confirm their state and exposed ports.

Command Used:

```
sudo docker compose ps
```

What Is Being Done: The student confirms the backend and database containers are healthy and checks which ports each one is listening on.

Observed Result: The output shows laravel-backend (image laravel-3tier-backend) up and listening

on 9000/tcp, and laravel-db (image mysql:8.0) up and listening on 3306/tcp and 33060/tcp, both running for 12 minutes at the time of this check.



Figure 30: Verifying the backend and database containers are running with docker compose ps.

Part C: Verifying the Database, Network, and Public Access

Running Migrations Inside the Container

With the containers running, the Laravel database migrations — including the new students table — are applied inside the backend container itself.

Command Used:

```
sudo docker exec laravel-backend php artisan migrate --force
```

Command Explanation:

- docker exec laravel-backend <command> → runs a command inside the already-running laravel-backend container.
- php artisan migrate --force → applies all pending database migrations; --force allows the migration to run without an interactive confirmation prompt, which is necessary since this is a non-interactive production-style environment.

What Is Being Done: The student creates all of the application's database tables, including the new students table, inside the MySQL container, by running the migration command from inside the backend container.

Observed Result: All four migrations — create_users_table, create_cache_table, create_jobs_table, and create_students_table — report as DONE, confirming the database schema was created successfully.

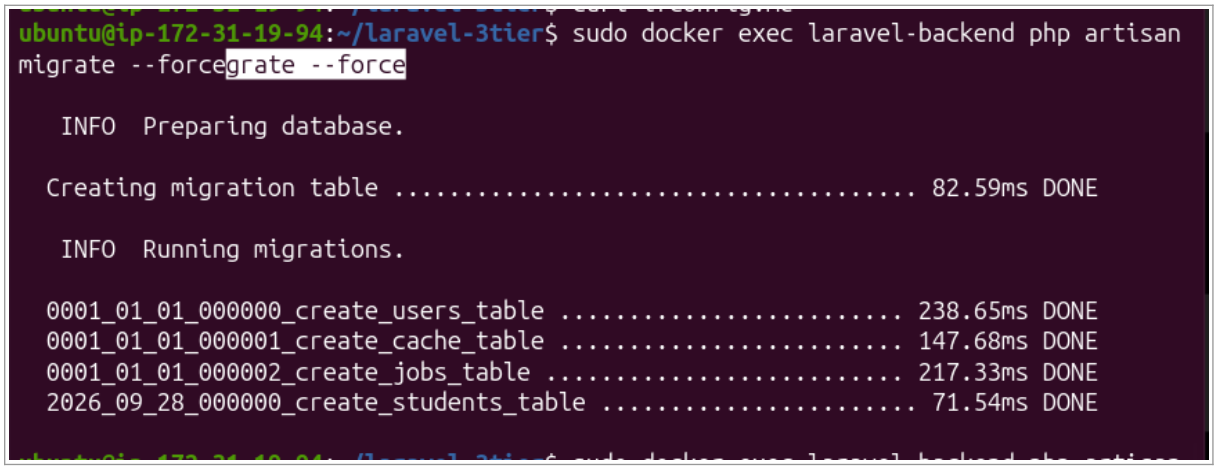


Figure 31: Running the Laravel database migrations inside the backend container.

The migration status is checked again for confirmation, and the Docker networks present on the host are listed to identify the network Compose created for this project.

Command Used:

```
sudo docker exec laravel-backend php artisan migrate:status
sudo docker network ls
```

Command Explanation:

- php artisan migrate:status → lists every migration and whether it has been run.
- docker network ls → lists all Docker networks on the host, along with their driver and scope.

What Is Being Done: The student double-checks that every migration, including the new students table, has actually been applied, then identifies the private network that Compose created for this project among the default Docker networks.

Observed Result: migrate:status confirms all four migrations show "Ran", and docker network ls lists the default bridge, host, and none networks alongside the project-specific laravel-3tier_laravel_network, which uses the bridge driver.

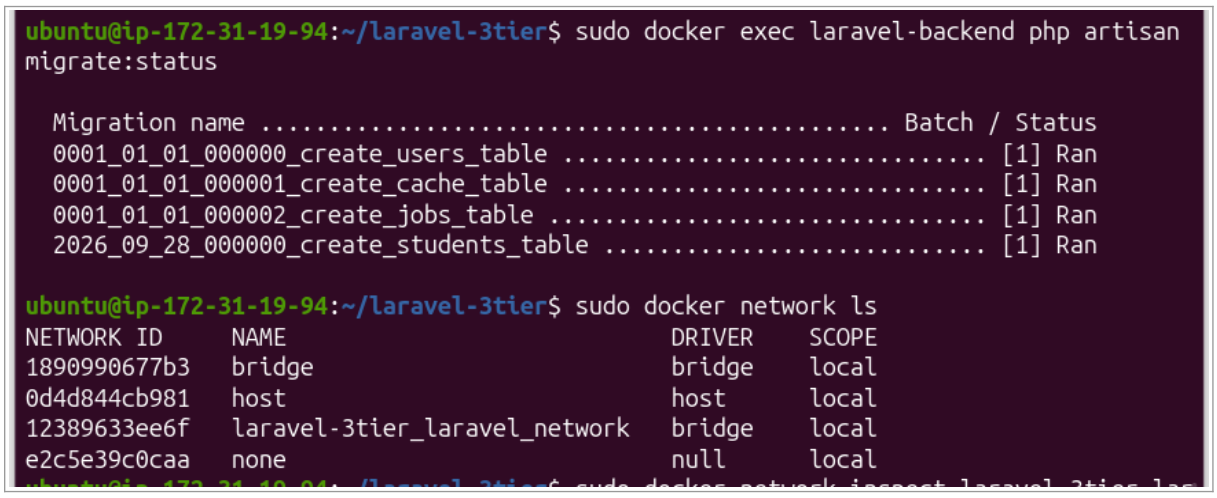


Figure 32: Confirming all migrations have run and identifying the project's Docker network.

Inspecting the Docker Network

The project's Docker network is inspected in detail to see its configuration.

Command Used:

```
sudo docker network inspect laravel-3tier_laravel_network
```

Command Explanation:

- docker network inspect <name> → prints detailed configuration for a Docker network, including its driver, subnet, and gateway.

What Is Being Done: The student examines the private bridge network that lets the frontend, backend, and database containers communicate with each other while staying isolated from other Docker projects on the same host.

Observed Result: The network is confirmed as a local bridge network with IPv4 enabled, using subnet 172.18.0.0/16 and gateway 172.18.0.1 — the private address range containers on this project receive their internal IPs from.



Figure 33: Inspecting the Docker Compose network and its subnet configuration.

Verifying DNS Resolution Between Containers

Docker Compose automatically provides DNS resolution between containers using their service names, which is tested directly from inside the backend container.

Command Used:

```
sudo docker exec laravel-backend php -r "echo gethostbyname('db');"
```

Command Explanation:

- gethostbyname('db') → a PHP function that resolves a hostname to an IP address; here, 'db' is the service name given to the MySQL container in compose.yaml.

What Is Being Done: The student proves the backend container can find the database container purely by its Compose service name, without needing to know its IP address.

Observed Result: The command returns 172.18.0.2, confirming Docker's built-in DNS correctly resolves the service name "db" to the database container's internal IP address, which is exactly the mechanism Laravel's DB_HOST=db configuration relies on.

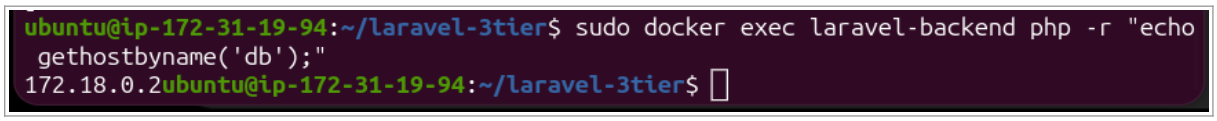


Figure 34: Resolving the hostname 'db' from inside the backend container, confirming internal DNS.

Confirming the Stack Is Reachable on Port 80

The stack is brought up again and re-checked, confirming the whole setup starts cleanly and consistently from the same Compose configuration.

Command Used:

```
sudo docker compose up -d
sudo docker compose ps
sudo docker network inspect laravel-3tier_laravel_network
```

What Is Being Done: The student re-runs the startup and verification commands to confirm the network, both database and application containers, and the frontend container all come up correctly together, with the frontend now confirmed to be listening on the public port.

Observed Result: Compose reports the network, laravel-db, laravel-backend, and laravel-frontend as created/started, and docker compose ps confirms all three containers are Up, with laravel-frontend specifically mapped from 0.0.0.0:80 and [::]:80 on the host to port 80 in the container — the mapping that exposes the site publicly.

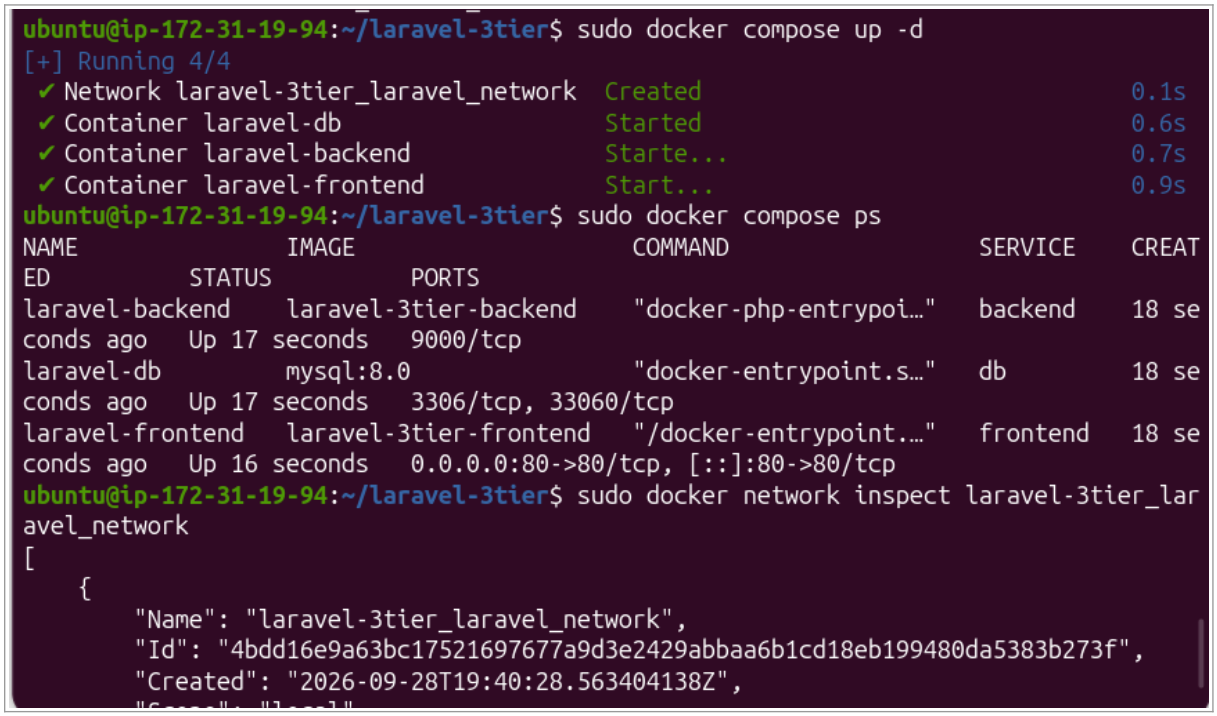


Figure 35: Re-confirming the full stack is running, with the frontend container mapped to the host's public port 80.

Testing the Application from the Public Internet

With the frontend container mapped to the EC2 instance's public port 80, the application is finally tested from a regular web browser over the public internet.

What Is Being Done: The student opens the EC2 instance's public IP address (13.51.101.222) directly in a browser, with no port number needed since the application is mapped to the standard HTTP port 80.

Observed Result: The "Student Management System" page loads successfully over the public internet, showing the add-student form and a table already containing one record ("Sami Khan"). This confirms the entire chain — EC2 security group, Nginx, PHP-FPM, and MySQL — is working correctly end-to-end.

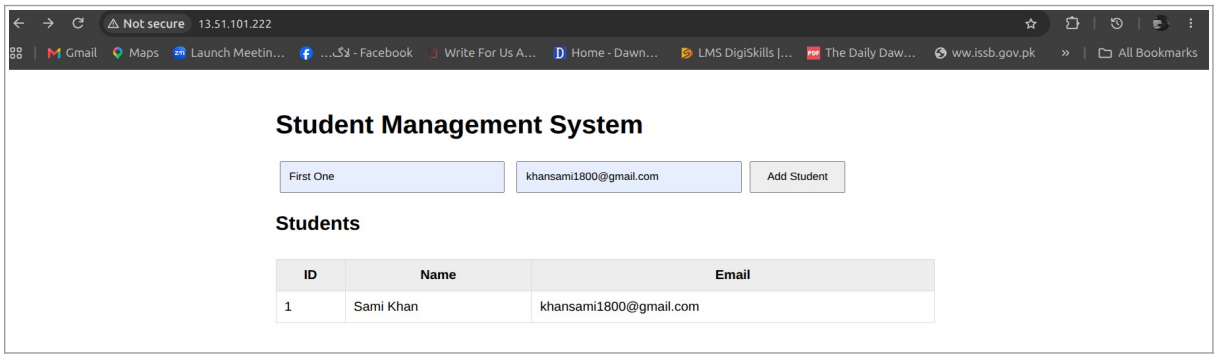


Figure 36: The Student Management System loaded from a browser at the EC2 instance's public IP address.

Verifying the Database Schema

The database is queried directly to confirm the students table was created correctly alongside Laravel's standard tables.

Command Used:

```
sudo docker exec laravel-db mysql -ularavel -ppassword laravel -e "SHOW TABLES;"
```

Command Explanation:

- docker exec laravel-db mysql -ularavel -ppassword laravel -e "SHOW TABLES;" → connects to MySQL inside the laravel-db container as the laravel user and lists every table in the laravel database.

What Is Being Done: The student confirms the students table exists in the database, alongside Laravel's built-in tables, directly at the database level rather than through the application.

Observed Result: The output lists cache, cache_locks, failed_jobs, job_batches, jobs, migrations, password_reset_tokens, sessions, students, and users — confirming the students table was created successfully by the earlier migration.



Figure 37: Confirming the students table exists using SHOW TABLES inside the database container.

The same check is repeated a second time as a final confirmation before continuing.

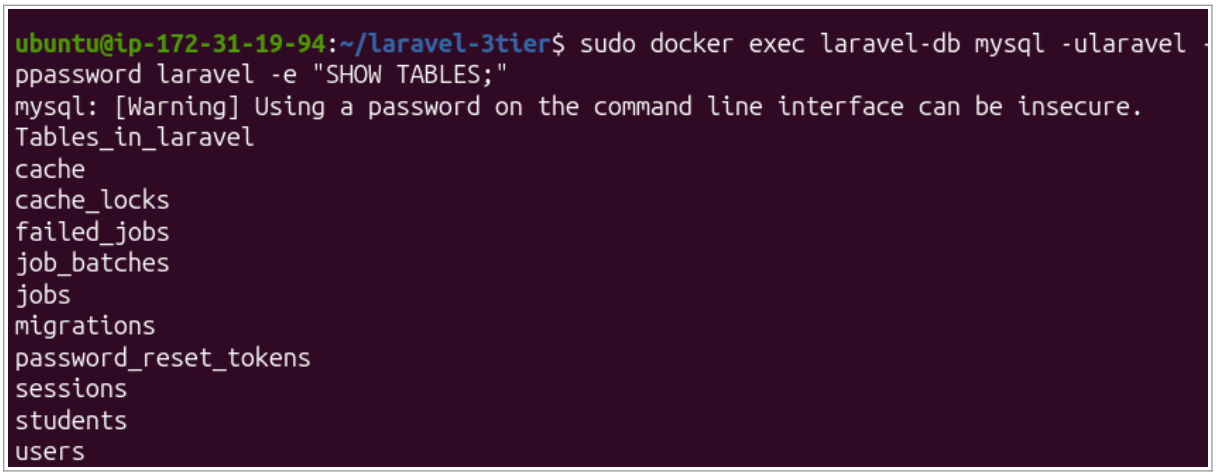


Figure 38: Re-confirming the students table with a second SHOW TABLES check.

Inspecting Docker Volumes

The named Docker volumes backing the stack's persistent data are listed and inspected, to understand exactly where their data physically lives on the host.

Command Used:

```
sudo docker volume ls
sudo docker volume inspect laravel-3tier_backend_data
sudo docker volume inspect laravel-3tier_db_data
```

Command Explanation:

- docker volume ls → lists every Docker volume on the host.
- docker volume inspect <name> → shows detailed information about a volume, including which Compose project it belongs to and its mount point on the host filesystem.

What Is Being Done: The student identifies the two named volumes Compose created for this project — one for the backend's application files and one for the database's data — as a first step before testing that data in these volumes survives a container being deleted.

Observed Result: Two project volumes are found, laravel-3tier_backend_data and laravel-3tier_db_data, each labelled with the laravel-3tier Compose project and mounted from /var/lib/docker/volumes/ on the host.

```
ubuntu@ip-172-31-19-94:~/laravel-3tier$ sudo docker volume ls
DRIVER    VOLUME NAME
local     b7be434589c57a11ca14ea9514ece29543cf5b4d7c167ec7066ead7333346b60
local     bbb747a942ab82407ddc9235680ddc79a80fe77a443f10cde2a9a613282efa23
local     laravel-3tier_backend_data
local     laravel-3tier_db_data
ubuntu@ip-172-31-19-94:~/laravel-3tier$ sudo docker volume inspect laravel-3tier_back
end_data
sudo docker volume inspect laravel-3tier_db_data
[
    {
        "CreatedAt": "2026-09-28T19:16:15Z",
        "Driver": "local",
        "Labels": {
            "com.docker.compose.config-hash": "33638627489b20dece0711fcb6a534db7dfc82
e553be6659641da444d56ce320",
            "com.docker.compose.project": "laravel-3tier",
            "com.docker.compose.version": "2.40.3",
            "com.docker.compose.volume": "backend_data"
        },
        "Mountpoint": "/var/lib/docker/volumes/laravel-3tier_backend_data/_data",
```

Figure 39: Listing and inspecting the named Docker volumes used by the backend and database services.

Confirming the Submitted Record Was Saved

Finally, the students table is queried again to confirm the record submitted earlier through the browser form was actually written to the database.

Command Used:

```
sudo docker exec laravel-db mysql -ularavel -ppassword laravel -e "SELECT * FROM students;"
```

What Is Being Done: The student verifies, directly at the database level, that the form submission from the browser test earlier was correctly saved.

Observed Result: The query returns one row: id 1, name “Sami Khan”, email “khansami1800@gmail.com”, with created_at and updated_at timestamps, confirming the form submission reached the backend container and was written to the database container successfully.

```
ubuntu@ip-172-31-19-94:~/laravel-3tier$ sudo docker exec laravel-db mysql -ularavel -
ppassword laravel -e "SELECT * FROM students;"
mysql: [Warning] Using a password on the command line interface can be insecure.
id      name    email   created_at      updated_at
1       Sami Khan       khansami1800@gmail.com  2026-09-28 19:43:29     2026-09-28 19
:43:29
ubuntu@ip-172-31-19-94:~/laravel-3tier$
```

Figure 40: Confirming the student record submitted through the browser was saved to the database.

Part D: Testing Docker Volume Persistence

Inspecting Container Volume Mounts and Recreating the Database Container

To prove that the containers' data is stored in the named Docker volumes and not inside the containers themselves, the exact mount points are inspected, the existing data is confirmed once more, and the database container is then deliberately deleted and recreated.

Command Used:

```
sudo docker inspect laravel-backend --format '{{json .Mounts}}'
sudo docker inspect laravel-db --format '{{json .Mounts}}'
sudo docker exec laravel-db mysql -ularavel -ppassword laravel -e "SELECT * FROM students;"
sudo docker rm -f laravel-db
sudo docker compose up -d db
```

Command Explanation:

- docker inspect <container> --format '{{json .Mounts}}' → prints just the mount information for a container, in JSON, showing which volume is attached and where it is mounted inside the container.
- docker rm -f laravel-db → forcefully stops and permanently deletes the laravel-db container.
- docker compose up -d db → recreates only the db service from compose.yaml as a brand-new container, re-attaching it to the same named volume.

What Is Being Done: The student first confirms exactly which volume is mounted where for both the backend (laravel-3tier_backend_data at /var/www/html) and the database (laravel-3tier_db_data at /var/lib/mysql), re-confirms the existing student record, and then intentionally destroys the database container — which would normally wipe out any data stored only inside that container — before immediately recreating it from the same Compose definition.

Observed Result: The inspect commands confirm each volume's exact mount point, the SELECT query confirms the Sami Khan record still exists just before the container is deleted, the old container is removed successfully, and Compose reports a fresh laravel-db container as Started, ready to be checked again.



Figure 41: Inspecting volume mount points, confirming existing data, then removing and recreating the

database container.

Confirming Images Remain Available After Recreation

After recreating the database container, the local image list is checked once more to confirm the images used by the stack were not affected by removing the container.

Command Used:

```
sudo docker images
```

What Is Being Done: The student confirms that deleting and recreating the laravel-db container did not remove or affect the underlying mysql:8.0 image, nor the backend and frontend images — only the container instance was replaced.

Observed Result: The image list still shows laravel-3tier-backend:latest, laravel-3tier-frontend:latest, and mysql:8.0, each marked as "In Use", confirming the images remained available locally and the new laravel-db container was able to start instantly from the existing mysql:8.0 image without needing to re-pull anything, while its actual data continues to live in the separate, untouched laravel-3tier_db_data volume.

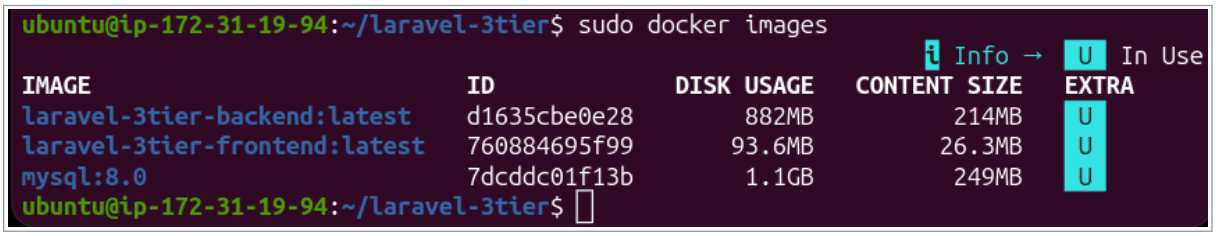


Figure 42: Confirming the underlying images remain available locally after the database container was recreated.

Part E: Publishing the Images to Docker Hub

As a final step, the custom backend and frontend images are published to Docker Hub, making them available to be pulled and deployed on any other machine without rebuilding them from source.

Logging in to Docker Hub

Before any image can be pushed, the student must authenticate with Docker Hub.

Command Used:

```
sudo docker login
```

Command Explanation:

- docker login → authenticates the local Docker client with Docker Hub (or another configured registry), using previously saved credentials if available.

What Is Being Done: The student logs in to their Docker Hub account (username sami1800) from the EC2 instance, which is required before any image can be pushed to that account's repositories.

Observed Result: Docker reports it is authenticating with existing credentials for user sami1800 and

confirms “Login Succeeded”.

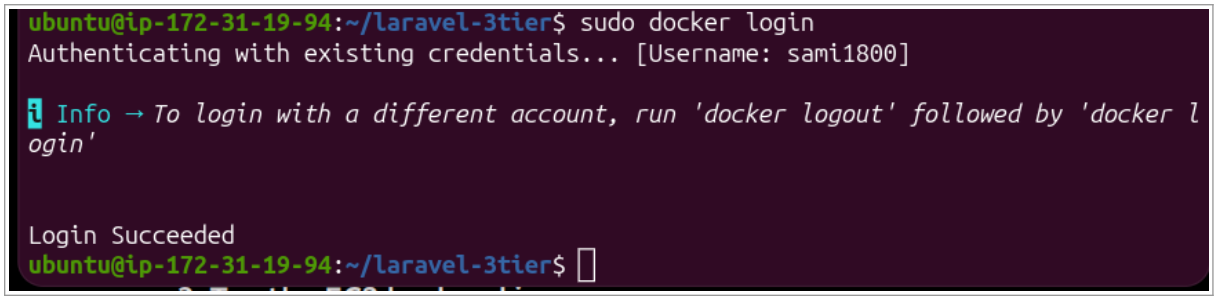


Figure 43: Logging in to Docker Hub from the EC2 instance.

Pushing the Backend Image

The backend image is given a Docker Hub-style tag and pushed to the registry.

Command Used:

```
sudo docker tag laravel-3tier-backend:latest sami1800/laravel-backend-ec2:1.0
sudo docker push sami1800/laravel-backend-ec2:1.0
```

Command Explanation:

- docker tag <local-image> <repository>:<tag> → creates an additional name for an existing local image, in the <docker-hub-username>/<repository-name>:<tag> format required to push to a specific Docker Hub repository.
- docker push <repository>:<tag> → uploads the tagged image’s layers to Docker Hub.

What Is Being Done: The student prepares the locally built backend image for publishing by tagging it as sami1800/laravel-backend-ec2:1.0, then uploads it to Docker Hub.

Observed Result: Docker Hub accepts the push and begins uploading each image layer, reporting “Pushed” for each one in turn.

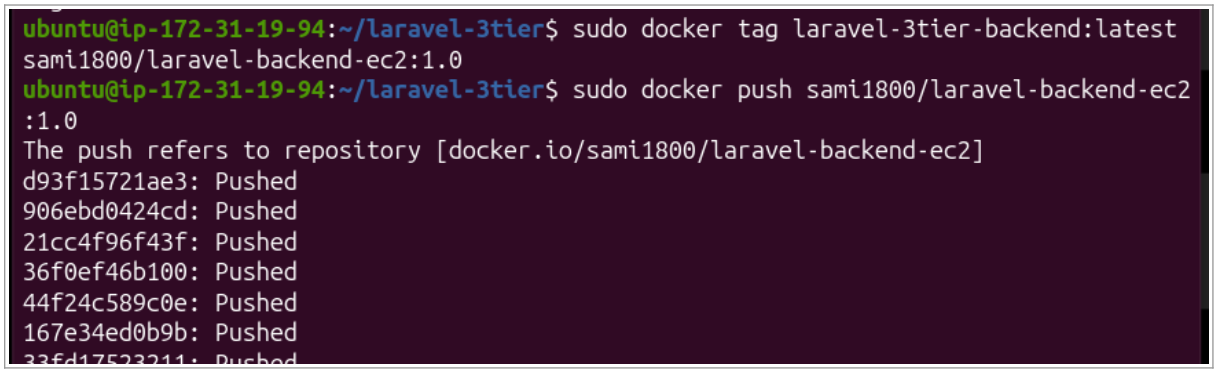


Figure 44: Tagging and pushing the backend image to Docker Hub as sami1800/laravel-backend-ec2:1.0.

The push completes, with Docker Hub confirming the final image digest and size for the uploaded backend image.



Figure 45: The backend image push completing successfully, with its digest confirmed.

Pushing the Frontend Image

The frontend image is tagged and pushed the same way.

Command Used:

```
sudo docker tag laravel-3tier-frontend:latest sami1800/laravel-frontend-ec2:1.0
sudo docker push sami1800/laravel-frontend-ec2:1.0
```

What Is Being Done: The student publishes the frontend image to the same Docker Hub account, so that both custom images needed to run the stack are available remotely.

Observed Result: Docker Hub accepts the push and uploads each layer of the frontend image, reporting “Pushed” for each one.



Figure 46: Tagging and pushing the frontend image to Docker Hub as sami1800/laravel-frontend-ec2:1.0.

The frontend push also completes successfully, with Docker Hub confirming its final digest and size.



Figure 47: The frontend image push completing successfully, with its digest confirmed.

Verifying the Published Images

As a final confirmation, the locally tagged images and the Docker Hub web interface are both checked to confirm the images were published correctly.

Command Used:

```
sudo docker images | grep sami1800
```

Command Explanation:

- docker images | grep sami1800 → filters the local image list down to just the images tagged under the sami1800 Docker Hub namespace.

What Is Being Done: The student confirms locally that both the backend and frontend images were tagged correctly under their Docker Hub account, alongside a MySQL image also tagged for this account.

Observed Result: Three images are listed: sami1800/laravel-backend-ec2:1.0, sami1800/laravel-frontend-ec2:1.0, and sami1800/laravel-mysql-ec2:8.0, confirming all three were correctly tagged under the sami1800 namespace.

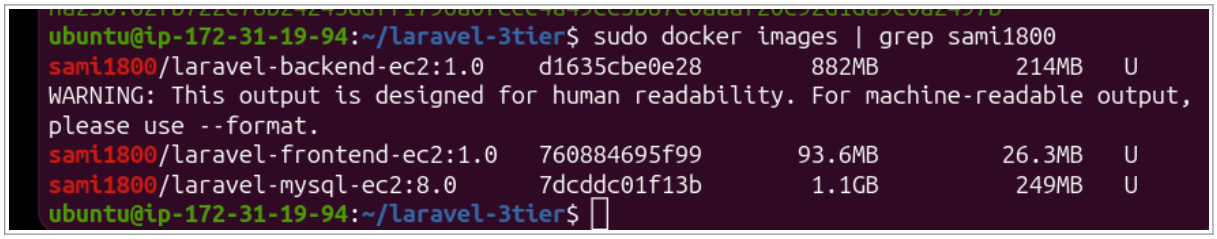


Figure 48: Confirming all three images are correctly tagged under the sami1800 Docker Hub namespace.

Finally, the Docker Hub website itself is opened to confirm the repositories are live and publicly accessible.

Observed Result: The Docker Hub "Repositories" page for the sami1800 account lists all three repositories — laravel-mysql-ec2, laravel-frontend-ec2, and laravel-backend-ec2 — each marked Public and showing a recent "last pushed" time, confirming the images were published successfully and are available for anyone to pull.

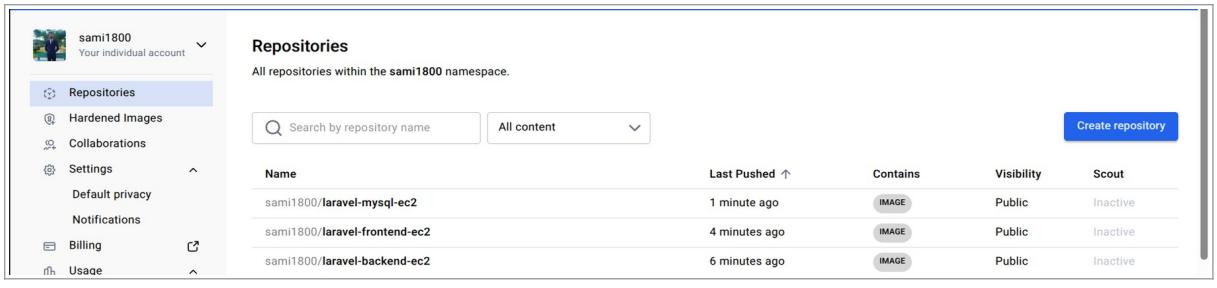


Figure 49: The Docker Hub web interface confirming all three repositories were published successfully.

Conclusion

This lab extended the standard three-tier Laravel/Docker workflow onto a real cloud server: an AWS

EC2 instance running Ubuntu with Docker already installed. The Student Management application was scaffolded and configured directly on the host, packaged into custom backend and frontend Docker images, and orchestrated with Docker Compose alongside a MySQL database container. The stack was verified end-to-end over the public internet at the instance's public IP address on port 80, its internal Docker network and service-name DNS resolution were inspected directly, and its named Docker volumes were proven to keep database and application data safe even when the database container itself was deleted and recreated. Finally, both custom images were published to Docker Hub, making the application portable and ready to be deployed on any other Docker host without being rebuilt from source. Together, these steps demonstrate practical, production-relevant DevOps skills: cloud deployment, multi-container orchestration, container networking, persistent storage, and image distribution.